

Esame 20260219

Esercizio 3

(1) Esercizio 3 v1

ESSAY

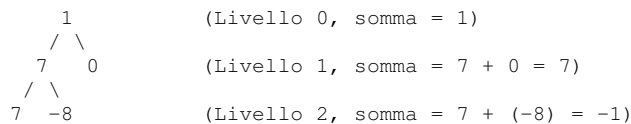
marked out of 10

penalty 0

File picker

In un albero binario, i nodi sono organizzati in "livelli". La radice si trova al livello 0, i suoi figli al livello 1, i figli dei figli al livello 2, e così via. Per un albero binario di interi, la "somma di livello" é definita come la somma di tutti i valori dei nodi che si trovano allo stesso livello (o profondità).

Ad esempio, dato il seguente albero binario:



la somma dei nodi al livello 0 é 1, al livello 1 é $7 + 0 = 7$, e al livello 2 é $7 + (-8) = -1$.

Data la libreria `treequeue` che implementa sia un albero binario di interi che una coda di nodi di un albero binario (file `treequeue.h` e `treequeue.cpp`), implementare la dichiarazione e la definizione della funzione `calcola`.

La funzione `calcola` prende come primo argomento un puntatore ad un nodo di un albero binario (di tipo `Node*`), come secondo argomento un intero passato per riferimento che dovrà contenere il valore **massimo** fra tutte le somme degli interi nello stesso livello, e come terzo argomento un intero passato per riferimento che dovrà contenere il livello dell'albero per cui si ottiene la somma corrispondente al secondo argomento (a parità di somma si deve restituire il livello più vicino alla radice, ovvero quello con indice minore).

Se l'albero é vuoto, la funzione dovrà impostare il secondo argomento a 0 ed il terzo a -1.

Di seguito é riportato un esempio di esecuzione del programma (l'albero viene stampato in orizzontale).

```
computer > ./a.out
Stampa dell'albero:
0
1
  -8
7
  7
Somma livello 0: 1
Somma livello 1: 7
Somma livello 2: -1
Somma richiesta: 7 al livello: 1
-----
L'albero e' vuoto.
-----
-6
1
  -8
7
  7
Somma livello 0: 1
Somma livello 1: 1
Somma livello 2: -1
Somma richiesta: 1 al livello: 0
-----
-8
1
  -8
7
```

```
7
Somma livello 0: 1
Somma livello 1: -1
Somma livello 2: -1
Somma richiesta: 1 al livello: 0
```

Suggerimenti: Si suggerisce di utilizzare una coda di nodi per implementare un algoritmo di visita in ampiezza (breadth-first) dell'albero binario, in modo da poter calcolare la somma dei nodi a ciascun livello in modo efficiente.

- inserire la radice dell'albero nella coda, e iterare finché la coda non é vuota (questo rappresenta i nodi a livello 0);
- fintanto che la coda non é vuota:
 - calcolare la dimensione della coda (rappresenta i nodi al livello corrente);
 - calcolare la somma dei nodi presenti in questo livello (usando un ciclo limitato dal size della coda a questo livello), aggiungendo gli eventuali figli nella coda (per il livello successivo);
 - confrontare la somma calcolata con il valore trovato finora, aggiornando di conseguenza il secondo e terzo argomento della funzione. La parte critica è distinguere quando finisce un livello e inizia il successivo (dimensione della coda a inizio del ciclo).

La coda ha una capacità massima specificata in fase di inizializzazione (con la funzione `createQueue`), dovrà quindi avere una capacità sufficiente per contenere tutti i nodi necessari al calcolo di quanto richiesto, e indipendente dagli esempi considerati (e.g., la dimensione dell'albero).

Note:

- **In caso di errori di compilazione della parte fornita, segnalare tempestivamente il problema al docente in aula per trovare una soluzione comune a tutti.**
- Scaricare i file `esercizio3.cpp`, `treequeue.cpp`, e `treequeue.h`. Modificare il solo file `esercizio3.cpp` per inserire il codice necessario per rispondere a questo esercizio. **Caricare il file sorgente risultato delle vostre modifiche a soluzione di questo esercizio** nello spazio apposito.
- All'interno di questo programma **non è ammesso** l'utilizzo di variabili globali o di tipo `static` e di funzioni di libreria al di fuori di quelle definite in `iostream` e `cstdlib`.
- Si ricorda che, gli esempi di esecuzione sono puramente indicativi, e la soluzione proposta **NON** deve funzionare solo per l'input fornito, ma deve essere robusta a variazioni compatibili con la specifica riportata in questo testo.
- Si ricorda di inserire solo nuovo codice e di **NON MODIFICARE** il resto del programma (pena annullamento dell'esercizio).
- Si ricorda che la soluzione deve essere implementata in C++ **NON usando altri elementi della C++ standard template library** tranne le funzioni definite nelle librerie concesse, anche se il file compila senza cambiare gli header!

`esercizio3.cpp`

`treequeue.cpp`

`treequeue.h`

Information for graders:

(2) Esercizio 3 v2

ESSAY

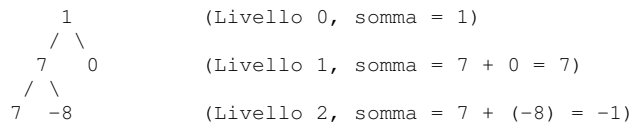
marked out of 10

penalty 0

File picker

In un albero binario, i nodi sono organizzati in "livelli". La radice si trova al livello 0, i suoi figli al livello 1, i figli dei figli al livello 2, e così via. Per un albero binario di interi, la "somma di livello" é definita come la somma di tutti i valori dei nodi che si trovano allo stesso livello (o profondità).

Ad esempio, dato il seguente albero binario:



la somma dei nodi al livello 0 é 1, al livello 1 é $7 + 0 = 7$, e al livello 2 é $7 + (-8) = -1$.

Data la libreria `treequeue` che implementa sia un albero binario di interi che una coda di nodi di un albero binario (file `treequeue.h` e `treequeue.cpp`), implementare la dichiarazione e la definizione della funzione `calcola`.

La funzione `calcola` prende come primo argomento un puntatore ad un nodo di un albero binario (di tipo `Node*`), come secondo argomento un intero passato per riferimento che dovrà contenere il valore **minimo** fra tutte le somme degli interi nello stesso livello, e come terzo argomento un intero passato per riferimento che dovrà contenere il livello dell'albero per cui si ottiene la somma corrispondente al secondo argomento (a parità di somma si deve restituire il livello più vicino alla radice, ovvero quello con indice minore).

Se l'albero é vuoto, la funzione dovrà impostare il secondo argomento a 0 ed il terzo a -1.

Di seguito é riportato un esempio di esecuzione del programma (l'albero viene stampato in orizzontale).

```
computer > ./a.out
Stampa dell'albero:
0
1
  -8
  7
    7
Somma livello 0: 1
Somma livello 1: 7
Somma livello 2: -1
Somma richiesta: -1 al livello: 2
-----
L'albero e' vuoto.
-----
-6
1
  -8
  7
    7
Somma livello 0: 1
Somma livello 1: 1
Somma livello 2: -1
Somma richiesta: -1 al livello: 2
-----
-8
1
  -8
  7
    7
Somma livello 0: 1
Somma livello 1: -1
Somma livello 2: -1
Somma richiesta: -1 al livello: 1
```

Suggerimenti: Si suggerisce di utilizzare una coda di nodi per implementare un algoritmo di visita in ampiezza (breadth-first) dell'albero binario, in modo da poter calcolare la somma dei nodi a ciascun livello in modo efficiente.

- inserire la radice dell'albero nella coda, e iterare finché la coda non è vuota (questo rappresenta i nodi a livello 0);
- fintanto che la coda non è vuota:
 - calcolare la dimensione della coda (rappresenta i nodi al livello corrente);
 - calcolare la somma dei nodi presenti in questo livello (usando un ciclo limitato dal size della coda a questo livello), aggiungendo gli eventuali figli nella coda (per il livello successivo);
 - confrontare la somma calcolata con il valore trovato finora, aggiornando di conseguenza il secondo e terzo argomento della funzione. La parte critica è distinguere quando finisce un livello e inizia il successivo (dimensione della coda a inizio del ciclo).

La coda ha una capacità massima specificata in fase di inizializzazione (con la funzione `createQueue`), dovrà quindi avere una capacità sufficiente per contenere tutti i nodi necessari al calcolo di quanto richiesto, e indipendente dagli esempi considerati (e.g., la dimensione dell'albero).

Note:

- **In caso di errori di compilazione della parte fornita, segnalare tempestivamente il problema al docente in aula per trovare una soluzione comune a tutti.**
- Scaricare i file `esercizio3.cpp`, `treequeue.cpp`, e `treequeue.h`. Modificare il solo file `esercizio3.cpp` per inserire il codice necessario per rispondere a questo esercizio. **Caricare il file sorgente risultato delle vostre modifiche a soluzione di questo esercizio** nello spazio apposito.
- All'interno di questo programma **non è ammesso** l'utilizzo di variabili globali o di tipo `static` e di funzioni di libreria al di fuori di quelle definite in `iostream` e `cstdlib`.
- Si ricorda che, gli esempi di esecuzione sono puramente indicativi, e la soluzione proposta NON deve funzionare solo per l'input fornito, ma deve essere robusta a variazioni compatibili con la specifica riportata in questo testo.
- Si ricorda di inserire solo nuovo codice e di **NON MODIFICARE** il resto del programma (pena annullamento dell'esercizio).
- Si ricorda che la soluzione deve essere implementata in C++ **NON usando altri elementi della C++ standard template library** tranne le funzioni definite nelle librerie concesse, anche se il file compila senza cambiare gli header!

`esercizio3.cpp`

`treequeue.cpp`

`treequeue.h`

Information for graders:

Total of marks: 20